

模块四: 数据工程 (上)

第十四讲: 数据采集与清洗——用爬虫获取实时数据

[解释]

第十四讲概览

本讲是数据工程模块的第二讲, 承接 L13 的"数据清洗与分析", 补全链路中最前端的环节——数据采集。实战场景是: 从 xuanxiao.org 网站爬取 QS、THE、ARWU、US News 四大全球大学排名系统的数据, 然后做多源清洗、对齐和交叉分析。

全程采用人机协同模式: 爬虫代码由 AI 编写, 研究者负责审核选择器逻辑、处理多源命名冲突、以及做出 SSOT (单点事实源) 的学术决策。

数据采集与清洗 多源数据整合

一、课程承接：从历史归档到实时采集

数据工程链路的第一步

[解释]

第一章：课程承接

本章建立 L13 与 L14 之间的衔接关系。L13 使用的是 Kaggle 上的历史归档数据（单一来源、静态快照），本讲的起点更靠前——解决"数据从哪里来"的问题，并引入"多源数据冲突"这个新挑战。

本章包含 3 页内容：从历史数据到实时数据、数据工程四步链路、本讲任务地图。

从历史数据到实时数据

上一讲，我们基于 B 站 1 亿用户的 **历史归档** 完成了完整的 EDA 分析。但那份数据有一个天然的局限。

L13 的数据来源

- Kaggle 上的 `.csv` 文件
- 数据采集于 2020 年，是一份**静态快照**
- 无法反映 2025 年的最新情况

真实研究中的常见困境

- 研究需要最新数据，但 Kaggle 上未必有
- 数据分散在各个网站的 HTML 页面中
- 没有现成的 `.csv` 下载——必须自己采集

💡 上一讲的起点是"拿到数据后怎么处理"。本讲的起点更靠前一步：**数据从哪里来**。

[解释]

静态数据集 vs 实时采集

Kaggle 之类的平台提供的是别人清洗好的 `.csv` 文件，属于"静态数据集"——它只反映采集时刻的状态，无法自动更新。而真实的研究场景中，数据往往分散在各种网页上，需要研究者自己编写程序去提取。这个过程就是"数据采集"，也叫"爬虫"（Web Scraping）。

本讲的定位是：补全数据工程链路中"采集"这个环节，让研究者具备从零获取数据的能力。

数据工程全链路

数据从原始状态变成有价值的结论，需要经过四个环节。

完整链路

- 1. 采集 (Collection): 从网页、API 获取原始数据
- 2. 清洗 (Cleaning): 去除脏数据、统一格式
- 3. 分析 (Analysis): 统计、可视化、建模
- 4. 结论 (Insight): 提炼学术或业务洞察

两讲的对比

	L13	L14
起点	清洗（数据现成）	采集（数据自取）
数据源	1 个（Kaggle）	4 个（网页爬取）
挑战	脏数据	多源冲突

本讲目标：从"采集"开始走完全链路，并学会处理多个数据源之间的冲突。

[解释]

数据工程四步链路

采集→清洗→分析→结论是数据工程的标准流程。上一讲跳过了"采集"直接从"清洗"开始，是因为教学上需要先让大家掌握 EDA 的基本功。但在真实项目中，数据不会自动出现在硬盘里——研究者需要自己去获取。本讲的另一个新维度是"多源"：同一个研究对象（大学排名），从四个不同来源采集数据后，数据之间存在冲突。如何处理这种冲突，是本讲的核心教学点。

本讲任务地图

我们将围绕一个真实场景——**全球大学排名**——走完数据工程的四个阶段。

Phase 1: 理解数据源

- 目标网站: xuanxiao.org (选校网)
- 聚合了 QS / THE / ARWU / US News 四大排名
- 每个排名的指标和评分体系各不相同

Phase 2: AI 协同爬虫

- 用自然语言指挥 AI 编写采集脚本
- 依次爬取四个排名系统的数据
- 输出 4 份结构化 `.csv` 文件

Phase 3: 多源数据清洗

- 统一大学名称 (实体对齐)
- 对齐不同排名系统的指标列
- 四表合并为一张综合分析表

Phase 4: EDA 交叉分析

- "同一所大学在四个榜单的排名差多少?"
- 揭示评价体系本身的差异与偏见
- 从数据中提炼出有价值的发现

[解释]

四大排名系统简介

全球大学排名有四个主流体系: **QS** (英国, 侧重学术声誉和雇主评价)、**THE** (英国泰晤士, 侧重教学和研究环境)、**ARWU** (中国软科, 侧重诺贝尔奖和高被引论文)、**US News** (美国, 侧重全球研究声誉和出版物)。由于评价指标不同, 同一所大学在不同榜单上的排名可能差异很大。这正好为"多源数据冲突"提供了天然的教学素材。

二、非结构化 Web 数据

网页 HTML 不是表格——但可以变成表格

[解释]

第二章：非结构化 Web 数据

本章帮助理解一个基本概念：网页上看到的整齐表格，在底层其实是 HTML 标签嵌套的非结构化数据。爬虫的工作就是从这些标签中提取文本，重新组织成结构化的表格。

本章包含 3 页内容：结构化 vs 非结构化数据的对比、xuanxiao.org 页面的 HTML 表格结构、四大排名系统的指标差异。

结构化数据 vs 非结构化数据

上一讲处理的 `.csv` 文件是**结构化数据**：行列清晰、字段固定。但网页上的数据不是这样的。

结构化数据（CSV）

```
rank,university,score
1,MIT,100
2,Cambridge,98.9
3,Oxford,96.9
```

- 每一行是一条记录
- 每一列是一个字段
- 程序可以直接读取和计算

非结构化数据（HTML）

```
<tr class="rankings-row">
  <td>1</td>
  <td>
    <a href="/uni/mit">MIT</a>
  </td>
  <td>100</td>
</tr>
```

- 数据藏在 HTML 标签之间
- 混杂了链接、样式、嵌套结构
- 程序不能直接读取——需要解析

💡 爬虫做的事情就一件：**把藏在 HTML 标签里的数据提取出来，变成整齐的表格。**

[解释]

HTML 与数据提取

HTML（超文本标记语言）是网页的基础格式。浏览器读取 HTML 后将其渲染为界面，但数据本身嵌套在各种标签（如 `<table>`，`<tr>`，`<td>`）之间。

爬虫的工作是用程序模拟浏览器的“阅读”过程，但不渲染页面，而是直接提取标签中的文本内容，按照行列关系组织成结构化数据。Python 中常用的工具是 `requests`（下载网页）+ `BeautifulSoup`（解析 HTML 标签）。

xuanxiao.org 的页面结构

以 QS 世界大学排名页面为例，观察数据在 HTML 中的位置。

页面上看到的

排名	大学名称	总分	学术声誉	雇主声誉
1	MIT	100	100	100
2	剑桥大学	98.9	100	100
3	牛津大学	96.9	100	100

HTML 中的结构

```
<table>
  <thead>
    <tr>
      <th>排名</th>
      <th>大学名称</th>
      <th>总分</th>
      <th>学术声誉</th>
      ...
    </tr>
  </thead>
  <tbody>
    <tr>
      <td>1</td>
      <td>Massachusetts Institute of ...</td>
      <td>100</td>
      ...
    </tr>
  </tbody>
</table>
```

对应的 URL

xuanxiao.org/rankings/qs/world-
university-rankings

核心认知：研究者不需要记住 HTML 语法。只需要能向 AI 描述："页面上有一个排名表格，请把它抓下来。"

[解释]

HTML 表格结构

HTML 中的表格由 `<table>` 标签定义。`<thead>` 包含表头（列名），`<tbody>` 包含数据行。每一行是 `<tr>`（table row），每一格是 `<td>`（table data）或 `<th>`（table header）。在 xuanxiao.org 的排名页面上，每个排名系统的数据都存放在一个 `<table>` 中。爬虫的核心任务就是定位到这个 table，然后逐行提取 td 中的文本。

四个数据源的指标差异

同一所大学在不同排名系统中的评价维度完全不同。

排名系统	核心指标	MIT 排名	URL 路径
QS	学术声誉、雇主声誉、国际化、可持续性	#1	/rankings/qs/world-university-rankings
THE	教学、研究环境、研究质量、产业收入	#2	/rankings/the/world-university-rankings
ARWU	诺贝尔奖校友、高被引学者、Nature/Science 论文	#4	/rankings/arwu/world-academic-ranking
US News	全球研究声誉、出版物、引用影响力	#2	/rankings/usnews/global-rankings

关键发现：MIT 在 QS 排第 1，但在 ARWU 排第 4。同一所大学，四个排名给出了四个不同的位置。**该信哪个？**——这就是多源数据冲突的真实场景。

[解释]

排名差异的根源

四大排名系统之所以给出不同结果，是因为它们的评价指标和权重不同。QS 的学术声誉指标占 30%，THE 的教学指标占 29.5%，ARWU 则完全基于客观量化指标（如诺贝尔奖数量）。这意味着没有一个排名是"绝对正确"的。每个排名反映的是其评价体系所侧重的维度。理解这一点是后续做多源分析的前提。

三、人机协同爬虫实战

指挥 AI 从网页中提取结构化数据

[解释]

第三章：人机协同爬虫实战

本章是动手环节。核心流程是：先用结构化 Prompt 指挥 AI 编写 QS 排名的爬虫脚本，审核代码逻辑后运行，然后把单源爬虫批量化扩展到四个排名系统。

本章包含 5 页内容：爬虫 Prompt 模板、AI 生成的代码解析、常见报错与排错、批量采集方法、采集伦理与 robots.txt。

爬虫 Prompt：给 AI 的第一条采集指令

和上一讲的 EDA Prompt 类似，爬虫指令也需要结构化地表达。

🎯 角色设定：你是一位有经验的 Python 数据工程师。

📁 目标网站：`https://xuanxiao.org/rankings/qs/world-university-rankings`，这是 QS 世界大学排名 2026 的完整数据页面。

📄 页面结构：页面中有一个 HTML `<table>` 表格，包含排名、大学名称、总分和各项指标得分。

🔧 采集任务：请用 `requests` + `BeautifulSoup` 编写爬虫脚本，提取表格中的所有数据，保存为 `qs_rankings.csv`。

⚠️ 附加要求：设置合理的 `User-Agent` 头，添加 1 秒延迟，代码须包含完整的中文注释。

💡 与 L13 的 Prompt 模板对比：结构完全一致——角色 + 数据背景 + 任务 + 约束。唯一的区别是数据来源从本地文件变成了网页。

[解释]

爬虫 Prompt 的关键要素

一个好的爬虫指令需要包含四个信息：**目标 URL**（抓哪个页面）、**页面结构描述**（数据在什么标签里）、**输出格式**（保存成什么文件）、**工程约束**（User-Agent、延迟等）。

其中“页面结构描述”是最重要的。如果只说“帮我爬这个网站”，AI 可能不知道该提取哪部分。但如果说“页面中有一个 HTML table，请提取其中的数据”，AI 就能精准定位。

AI 生成的爬虫代码

以 QS 排名页为例，AI 生成的采集脚本结构如下：

核心代码

```
import requests
from bs4 import BeautifulSoup
import pandas as pd
import time

url = "https://xuanxiao.org/rankings/qs/world-university-rankings"

# 伪装为正常浏览器访问
headers = {"User-Agent": "Mozilla/5.0 ..."}

# 下载网页内容
response = requests.get(url, headers=headers)
soup = BeautifulSoup(response.text, "html.parser")

# 定位排名表格
table = soup.find("table")
rows = table.find_all("tr")

# 逐行提取数据
data = []
for row in rows[1:]: # 跳过表头
    cols = row.find_all("td")
    data.append([col.text.strip() for col in cols])

# 保存为 CSV
df = pd.DataFrame(data, columns=["排名", "大学", "总分", ...])
df.to_csv("qs_rankings.csv", index=False)
```

代码审计要点

和 L13 一样，拿到 AI 的代码后先审计逻辑：

1. URL 对不对？ 确认是目标页面
2. 选择器对不对？
`soup.find("table")` 是否抓到了正确的表格
3. 列名对不对？ DataFrame 的列名是否与表头一致
4. 有没有设 User-Agent？ 否则可能被 403 拒绝
5. 有没有延迟？ 避免对服务器造成压力

 审计代码的方法和 L13 完全一致：看中文注释，理解加工顺序。

[解释]

requests + BeautifulSoup 工具链

`requests` 是 Python 的 HTTP 请求库，用于下载网页内容。`BeautifulSoup` 是 HTML 解析库，能把 HTML 文本转化为可查询的树结构。

`soup.find("table")` 会定位到页面中的第一个 `<table>` 标签。`find_all("tr")` 获取所有行，`find_all("td")` 获取某行中的所有列。`col.text.strip()` 提取标签内的纯文本并去除空格。这套组合是最基础的爬虫工具链。

常见报错与 AI 排错

爬虫代码的常见错误和处理方式：

典型报错

- **403 Forbidden**
 - 服务器拒绝访问。原因：没有设置 `User-Agent`
 - 修复：添加浏览器标识头
- **提取数据为空**
 - CSS 选择器没有匹配到目标元素
 - 修复：检查 HTML 结构，更新选择器
- **中文乱码**
 - 编码不匹配
 - 修复：`response.encoding = 'utf-8'`

AI 排错指令

和 L13 处理报错的方式完全一致：

"运行爬虫脚本时出现 403 错误，完整报错如下：[粘贴报错]。请查明原因并给出修复后的完整代码。"

"脚本运行成功但 CSV 文件是空的。目标页面的 URL 是 [URL]。请检查 HTML 结构并修正选择器。"

🔥 与 L13 一样：把报错信息原封不动地丢给 AI，让它诊断。

[解释]

爬虫常见错误的技术原因

403 Forbidden：许多网站会检查 HTTP 请求头中的 `User-Agent` 字段。如果这个字段为空（Python requests 的默认行为），服务器会判定为爬虫并拒绝访问。解决方法是在请求头中添加一个浏览器的标识字符串。

数据为空：通常是 `soup.find()` 没有匹配到元素。可能原因包括：表格用的不是标准 `<table>` 标签，或者页面通过 JavaScript 动态加载数据（此时需要用 Selenium 等工具）。

批量采集：从 1 个源到 4 个源

QS 的爬虫调通之后，接下来把它扩展到其他三个排名系统。

AI 指令

"请把刚才的 QS 爬虫脚本改造成可配置版本。用一个字典存储四个排名系统的 URL，依次爬取并分别保存为 CSV：

- QS: `.../qs/world-university-rankings`
- THE: `.../the/world-university-rankings`
- ARWU: `.../arwu/world-academic-ranking`
- US News: `.../usnews/global-rankings`

每次请求之间间隔 2 秒。"

产出

运行后得到 4 个文件：

<code>qs_rankings.csv</code>	(QS 排名)
<code>the_rankings.csv</code>	(THE 排名)
<code>arwu_rankings.csv</code>	(ARWU 排名)
<code>usnews_rankings.csv</code>	(US News 排名)

每个文件包含该排名系统的大学名称、综合排名、总分和各项指标。

💡 批量化的核心思路：先调通一个，再复制到多个。这和工程中的"先打样再量产"（L13 的 `.head(5)`）是同一个逻辑。

[解释]

从单次到批量的工程模式

这种"先调通一个、再批量化"的工作方式在软件工程中非常常见。先在单一目标上验证逻辑正确性（QS），确认无误后再参数化扩展到多个目标（THE/ARWU/USNews）。

代码中的核心改动是把硬编码的 URL 替换为一个字典或列表，然后用 for 循环遍历。每次循环中添加 `time.sleep(2)` 控制请求频率。

采集伦理：robots.txt 与访问礼仪

在爬取任何网站之前，需要了解两条基本规则。

robots.txt 协议

- 每个网站根目录下都有一个 `robots.txt` 文件
- 它声明了哪些页面允许爬虫访问、哪些禁止
- 查看方式：直接访问 `网站域名/robots.txt`
- 这不是法律，但业界共识是：必须遵守

工程礼仪

1. 控制频率：每次请求间隔 1-2 秒
2. 声明身份：设置合理的 User-Agent
3. 不采集隐私：不爬取用户个人信息
4. 适量采集：只取研究需要的数据

[解释]

robots.txt 与爬虫伦理

`robots.txt` 是网站管理员声明爬虫访问规则的标准文件，放在网站根目录下。它用 `User-agent` 指定规则适用的爬虫，用 `Disallow` 列出禁止访问的路径。

虽然 `robots.txt` 没有法律约束力，但不遵守它可能导致 IP 被封禁，甚至面临法律风险（特别是涉及个人数据时）。在教学场景中，我们只采集公开的排名数据，属于合理范围。

四、多源数据清洗与对齐

四份数据，四种格式——如何统一？

[解释]

第四章：多源数据清洗与对齐

本章处理采集后的核心难题：四份数据的列名不同、大学名称写法不同、评价指标也不同。需要依次完成实体对齐（统一名称）、指标对齐（统一 schema）、四表合并（outer join），最后用 SSOT 原则处理排名冲突。

本章包含 5 页内容：多源 EDA、实体对齐、指标对齐、四表合并、SSOT 原则。

四份数据的第一眼 EDA

拿到爬取结果后，先做一次快速体检——和 L13 的 EDA 流程完全一致。

AI 指令

"请分别读取 qs_rankings.csv、the_rankings.csv、arwu_rankings.csv、usnews_rankings.csv，对每份数据执行 `.describe()` 和 `.info()`，输出基础统计信息。"

常见发现

- 列名不同：QS 叫 Overall Score，THE 叫 总分
- 缺失值：部分排名有 - 或空白
- 行数不同：QS 有 1500 行，ARWU 有 1000 行

四份数据的列名对比

QS	THE	ARWU	US News
排名	排名	排名	排名
大学名称	大学名称	大学名称	大学名称
总分	总分	总分	总分
学术声誉	教学	获奖	全球研究声誉
雇主声誉	研究环境	高被引	出版物
...	...	...	...

[解释]

多源 EDA 的目的

单源 EDA（L13）的目的是检查数据质量；多源 EDA 的目的除了检查质量，还要发现**格式差异**。这些差异包括：列名不一致（需要重命名）、数据类型不一致（排名有的是数字有的是字符串）、缺失值用不同符号表示（有的用空值，有的用短横线）。只有先把这些差异摸清楚，后面的对齐和合并才有据可依。

实体对齐：统一大学名称

同一所大学在四个榜单上的名字可能不同。如果不统一，程序无法识别它们是同一个实体。

名称差异示例

QS	THE	ARWU
Massachusetts Institute of Technology	MIT	麻省理工学院
University of Cambridge	Cambridge	剑桥大学
Peking University	PKU	北京大学

AI 指令

"四份数据中同一所大学的名称写法不同。请建立一个统一映射字典，把所有名称对齐到英文全称（以 QS 的命名为基准）。然后更新其他三份数据的大学名称列。"

```
alias_map = {
    "MIT": "Massachusetts Institute of Technology",
    "麻省理工学院": "Massachusetts Institute of Technology",
    "Cambridge": "University of Cambridge",
    "剑桥大学": "University of Cambridge",
}
```

[解释]

实体对齐 (Entity Resolution)

实体对齐是数据工程中的常见问题：不同数据源对同一实体使用不同的名称。解决方法是建立一个**映射字典** (alias map)，把所有变体名称统一映射到一个标准名。选择哪个来源作为“标准”取决于研究需求。在本例中，QS 使用的英文全称覆盖面最广，因此作为基准。实际项目中，映射字典通常需要手动校验，因为自动匹配可能遗漏或误匹配。

指标对齐：只保留可比维度

四个排名系统的评价指标各不相同，无法直接合并。策略是只保留共有维度。

不可比的指标

- QS 有"雇主声誉"，其他三个没有
- ARWU 有"诺贝尔奖校友"，其他三个没有
- 这些列在合并时会产生大量空值

共有维度

所有排名系统都有两个共同字段：

- 综合排名（名次）
- 总分（满分 100）

统一 Schema

```
# 每份数据只保留三列
columns_to_keep = ["大学名称", "排名", "总分"]

# 重命名列以标识来源
qs = qs[columns_to_keep].rename(columns={
    "排名": "QS排名", "总分": "QS总分"
})
the = the[columns_to_keep].rename(columns={
    "排名": "THE排名", "总分": "THE总分"
})
# ARWU, USNews 同理...
```

[解释]

Schema 对齐策略

当多个数据源的列定义不同时，有两种策略：**并集合并**（保留所有列，缺失的填充值）和**交集合并**（只保留共有列）。前者信息保留完整但会有大量空值，后者简洁可比但会丢失部分信息。

在教学场景中选择交集策略（只保留排名和总分），是因为本讲的分析目标是“同一所大学在不同榜单的排名差多少”——这只需要名次数据就够了。

四表合并：Outer Join

名称和指标都对齐之后，用合并操作把四张表拼成一张。

代码

```
import pandas as pd

# 以大学名称为主键，四表合并
merged = qs.merge(the, on="大学名称", how="outer")
merged = merged.merge(arwu, on="大学名称", how="outer")
merged = merged.merge(usnews, on="大学名称", how="outer")

print(f"合并后行数: {len(merged)}")
print(merged.head(10))
```

合并结果示例

大学名称	QS 排名	THE 排名	ARWU 排名	USNews 排名
MIT	1	2	4	2
Cambridge	2	5	4	8

审计要点

- 使用 `outer` 而不是 `inner` ——保留所有大学，即使只出现在部分榜单
- 合并后检查行数是否合理
- 检查是否有大学只出现在 1-2 个榜单中（说明名称对齐可能不完整）

[解释]

Inner Join vs Outer Join

`inner join` 只保留在所有表中都出现的记录。`outer join` 保留所有记录，未匹配的字段填充空值。在多源分析中应该使用 `outer join`，因为“某所大学只出现在部分榜单”这个信息本身就有分析价值——它可能说明该大学在某些评价体系中不够格入选。如果用 `inner join`，这些差异信息会被直接丢掉。

SSOT：数据冲突时该信谁？

SSOT (Single Source of Truth) 即"单点事实源"：当多个来源的数据矛盾时，预先声明一个权威的参考标准。

冲突示例

MIT 在四个榜单的排名：

- QS: #1
- THE: #2
- ARWU: #4
- US News: #2

四个数字，四种说法。该信哪个？

SSOT 不是"选最好看的"

-  "QS 排名最高，所以 MIT 是第 1" → 事后挑选
-  "本研究以 QS 排名为主参考，其他排名用于交叉验证" → 事先声明

核心原则：在分析开始之前就声明优先级规则，而不是看到结果之后再挑选对自己有利的数据来源。

学术诚信：先定规则，再看数据。如果看到数据之后再选择标准，就不是研究，而是凑结论。

[解释]

SSOT（单点事实源）原则

SSOT 是数据工程中的核心概念：当多个数据源对同一事实给出不同说法时，必须指定唯一的权威来源作为最终参考。

在学术研究中，SSOT 的选择必须在方法论部分事先声明，不能事后更改。常见的选择依据包括：数据源的权威性（官方 vs 第三方）、数据的覆盖范围、数据的更新频率等。本讲中可以选择任何一个排名系统作为 SSOT，但必须给出选择理由。

五、EDA 价值验证与总结

数据分析能揭示什么？

[解释]

第五章：EDA 价值验证与总结

本章将合并后的数据用于交叉分析，计算每所大学在四个榜单排名的标准差（离散度），揭示不同评价体系对同一大学的看法差异。最后回顾完整链路并总结核心守则。

本章包含 4 页内容：排名离散度分析、完整回路复盘、核心守则、课堂收获与下一讲预告。

多源交叉分析：排名离散度

合并后的数据可以回答一个有趣的问题：**哪些大学在不同榜单中的排名最稳定？哪些波动最大？**

AI 指令

"请计算合并表中每所大学在四个榜单上排名的标准差（std），并按标准差排序。标准差越小说明排名越一致，越大说明不同评价体系对该校的看法差异越大。"

发现

- MIT 的离散度只有 1.3——四个榜单高度一致
- 一些大学的离散度超过 30——说明不同评价体系对它的看法截然不同
- **学术价值：**离散度高的大学值得深入研究——它们在什么指标上得分或失分？

分析示例

大学	QS	THE	ARWU	USNews	离散度 (std)
MIT	1	2	4	2	1.3
某大学	30	95	45	120	39.7

[解释]

排名离散度分析

标准差（Standard Deviation）衡量一组数字偏离平均值的程度。将它应用于一所大学在四个榜单上的排名，可以量化"不同评价体系对该大学的看法一致性"。

离散度低的大学（如 MIT）说明其综合实力在所有评价维度上都很强。离散度高的大学可能在某些维度（如研究产出）很强，但在另一些维度（如教学质量）较弱。分析离散度可以揭示排名背后评价体系的偏好差异。

完整回路：从采集到结论

回顾今天走过的完整链路。

数据工程流程

- 1. 选定数据源：xuanxiao.org 四大排名
- 2. AI 写爬虫：requests + BeautifulSoup
- 3. 采集数据：4 个排名系统 → 4 份 CSV
- 4. 实体对齐：统一大学名称
- 5. 指标对齐：只保留共有维度
- 6. 合并分析：Outer Join + 排名离散度
- 7. 得出结论：量化评价体系差异

与 L13 的对比

	L13	L14
数据来源	现成 CSV	网页爬取
数据源数量	1 个	4 个
新技能	EDA + 清洗	爬虫 + 多源对齐
共同点	代码由 AI 写，人负责审计和决策	

[解释]

两讲的能力递进

L13 建立了"人审计、AI 执行"的基本模式；L14 在此基础上增加了两项新能力：**数据采集**（从网页获取数据）和**多源对齐**（处理不同来源之间的格式与命名差异）。
从能力模型看，学完 L13+L14 之后，研究者已经具备了数据工程全链路的基础能力：能自己获取数据、清洗数据、分析数据、得出结论，全程由 AI 辅助执行。

核心守则复盘

采集守则

- 尊重 robots.txt
- 每次请求间隔 ≥ 1 秒
- 只采集研究所需的公开数据
- 声明 User-Agent

清洗与分析守则

- 先 EDA 再合并——确认列名和格式差异
- 使用 Outer Join——保留所有记录，不丢信息
- 预设 SSOT 优先级——在分析之前声明
- 先打样再量产——用 `.head(5)` 预览

贯穿两讲的核心原则：代码由 AI 写，但加工顺序的审计和数据取舍的决策，只能由研究者自己做。

[解释]

守则的工程意义

这些守则不是抽象的教条，每一条都是为了防止具体的错误：不设 User-Agent 会被 403；不做 EDA 就合并会遗漏格式差异；用 inner join 会丢失关键差异信息；不预设 SSOT 就容易滑入“挑选数据”的陷阱。在人机协同模式下，AI 能自动执行规则，但选择哪些规则、规则的优先级如何排列，仍然是研究者的决策。

课堂收获与下一讲预告

核心收获

- ☒ **数据采集**：学会用 AI 编写爬虫从网页获取数据
- ☒ **多源对齐**：掌握实体对齐（名称统一）和指标对齐（schema 统一）
- ☒ **冲突处理**：理解 SSOT 原则——先定规则再看数据
- ☒ **交叉分析**：用排名离散度揭示评价体系的差异

后续预告

L13 处理结构化数据，L14 采集结构化数据。但真实世界还有大量的非结构化文本——论文摘要、新闻报道——怎么处理？

下一讲：**非结构化文本处理——用 LLM 构建论文速读助手**

[解释]

本讲在课程体系中的位置

L13 建立了"单源数据的 EDA 和清洗"能力，L14 在此基础上新增了"数据采集"和"多源数据对齐"能力。两讲处理的都是结构化数据（数字、排名）。

下一讲（L15）将引入非结构化文本处理：通过 arXiv RSS 获取论文列表，调用硅基流动的 LLM API 进行分类和中文摘要，最终生成速读清单。这是数据工程模块从"可计算数据"到"需要理解的文本"的关键跨越。